

Aventura en el Reino de JS

Forma de trabajo

Se llevará a cabo en grupos de 4 personas, en caso de que haya picos se harán algunas parejas.

Esta actividad se desarrollará mediante la metodología de puzzle, de la siguiente manera:

- **Fase 1:** Se realizan los grupos y cada uno tiene asignado un módulo, deberá trabajar en él durante 30-60 minutos aproximadamente o hasta que se considere oportuno por el profesor.
- **Fase 2:** Se agruparán de forma momentánea las personas de todos los grupos que comparten el mismo módulo, con el fin de compartir sus ideas y la forma que han tenido de resolverlos, debatiendo cuál sería la forma más adecuada, óptima y limpia. Durante alrededor de 25 minutos (5 minutos de explicación por cada uno y otros 5 para preguntas entre todos).
- **Fase 3:** Cada integrante vuelve con su grupo inicial, y entre todos deberán fusionar sus módulos y explicar cada uno de ellos.

Cada grupo inicialmente creará un proyecto en github compartido, de él crearán una rama por cada integrante. Cada uno trabajará en su rama y en la fase 3, todas las ramas deberán llevarlas a la principal para ver el resultado final.

Objetivo principal

Se deberá hacer uso todo lo posible de la teoría vista en clase:

Clases, herencia, objetos, desestructuración, clonación, agrupación de datos y trabajo en módulos.

Crear la carpeta utils y el archivo de constantes en la integración final para mejorar el código.

Desarrollo del proyecto

Construir un mini-juego por consola:

- Jugadores con inventario y puntos
- Enemigos y jefes
- Un mercado de ítems para mejorar el equipo
- Un ranking final

Cada alumno desarrolla un **módulo independiente**, pero todos usan la misma estructura de datos para que luego puedan encajar.

Módulo de Jugadores

Qué representa:

Los personajes que van a jugar, con sus datos básicos y su inventario de objetos.

Qué debe contener:

- Una definición de “Jugador” con:
 - Nombre del jugador,
 - Puntos (su puntuación en el juego).
 - Un inventario de objetos (una lista con los objetos que vaya comprando).
 - Vida (se puede tener una propiedad adicional para indicar la vida máxima).
- Métodos o funciones que permitan:
 - Añadir un objeto al inventario (Habrà que clonar el de la tienda).
 - Sumar puntos al jugador cuando gane batallas.
 - Obtener ataque total: Calculará el ataque en función de los objetos.
 - Obtener defensa total: Calculará la defensa en función de los objetos.
 - Agrupar inventario por tipo de objeto.
 - Mostrar jugador: Se mostrará su nombre, puntos, vida, ataque, defensa e inventario.

Qué debe exportar para los demás:

- La clase Jugador.

Relación con otros módulos:

- Usará las funciones del **mercado** para que los jugadores puedan comprar objetos.
- Sus datos de puntos se usarán en el **ranking final**.

Módulo de Enemigos y Jefes

Qué representa:

Los oponentes que los jugadores tendrán que vencer.

Qué debe contener:

- Una definición de “Enemigo” con:
 - Tipo (enemigo o jefe)
 - Nombre del enemigo.
 - Nivel de ataque.
 - Puntos de vida.
- Una sub-clase “Jefe Final” que herede de Enemigo y añada:
 - Una habilidad especial.
 - Un multiplicador de daño, que por defecto tenga un valor.
- Un método de presentación que describa quién es el enemigo y, en el caso de los jefes, su habilidad.

Qué debe exportar para los demás:

- La clase Jugador y Jefe.

Relación con otros módulos:

- Los jugadores del **módulo de Jugadores** lucharán contra estos enemigos en las batallas del **módulo de Ranking y Batalla**.

Módulo de Mercado y Producto

Qué representa:

La tienda de objetos del juego donde los jugadores mejoran su equipo.

Qué debe contener:

- Una clase Producto con:
 - Nombre
 - Precio
 - Rareza (Común, rara, legendaria...)
 - Tipo (arma, armadura, consumible)
 - Bonus (aumenta ataque, defensa o cura vida)
- Con funciones para:
 - La presentación que describa al objeto, formateando su precio en € (los objetos guardan su precio sin decimales por ejemplo 950, esto se formateara a 9,50€).

- Aplicar un descuento: Le llega un valor y devuelve una copia de ese producto modificado.
- Un mercado que contenga una lista de productos.
- Con funciones para:
 - Filtrar productos por una rareza dada.
 - Aplicar un descuento a los productos que sean de un tipo o rareza concreta.
 - Buscar un producto por su nombre.
 - Mostrar un producto (se llamará al método que tiene este producto)

Qué debe exportar para los demás:

- La clase producto y sus funciones
- La lista mercado y sus funciones.

Relación con otros módulos:

- El **módulo de Jugadores** usará estas funciones para que sus personajes compren objetos y mejoren su inventario.

Módulo de Ranking y Batalla

Qué representa:

El sistema de combate y la clasificación final de los jugadores.

Qué debe contener:

- Una función “batalla” que reciba un jugador y un enemigo:
 - Calcule el resultado del combate (por ejemplo, si el jugador gana, suma puntos a su marcador).
- Una función para agrupar a los jugadores según su puntuación final:
 - “pro” para los que lleguen a una puntuación alta,
 - “rookie” para el resto.

- Mostrar el ranking final de todos los jugadores en una tabla por consola.

Qué debe exportar para los demás:

- Las funciones de batalla y de mostrar el ranking.

Relación con otros módulos:

- Recibe los **jugadores** y los **enemigos** de los otros módulos para simular las batallas.
- Usa la puntuación de los jugadores para generar el **ranking final**.

Integración final

Cuando todos los módulos estén listos:

1. Se crea un archivo principal que importe los módulos y sea el que organice el juego.
2. En ese archivo se simula una pequeña historia:
 - Los jugadores compran objetos en el **mercado**.
 - Cada jugador se enfrenta a uno o varios **enemigos**.
 - Se actualizan sus puntos.
3. Finalmente se ejecuta la función de **ranking** para mostrar la clasificación de los jugadores y ver quién es el campeón.